CAMPUS GENOME

THE STUDENT COLLECTIVE

PRODUCT & ENGINEERING UPGRADE

A campus with character. A backend with backbone.

An unofficial campus common room, full of student lore.
Gold, whimsy, and a more dependable backend.

FRONTEND + BACKEND + DELIVERY NOTES

42	2	0
automated checks passed	database engines verified	known npm advisories*

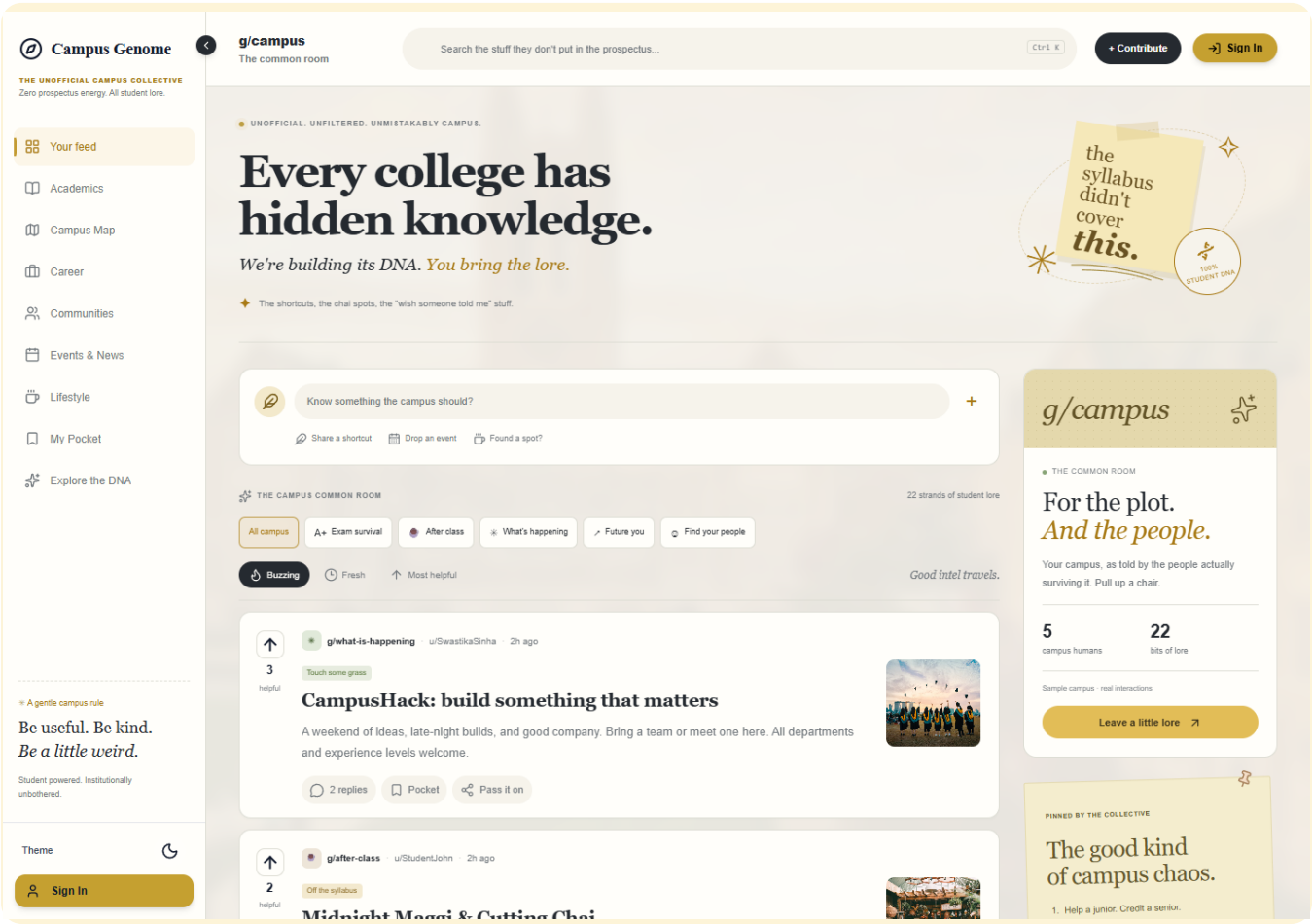
Built on the existing CampusGenome project. Existing campus data and UML models were preserved. This report explains the finished work, the choices behind it, and how to run it.

16 OCTOBER 2026

* npm audit results at delivery; advisories can change over time.

Off the prospectus. On campus.

A student-led forum feel, with clearer hierarchy and the original gold-and-serif personality.



A campus common room, with character.

Readable paper-like posts, useful voting rails, topic channels, student handles and flairs bring an unofficial forum rhythm. Gold accents, a serif hero, taped notes, hand-drawn details and a faint campus backdrop keep the whimsy.

Less clutter. More room for the good stuff.

The header is compact, search stays within reach, posts form one clear reading column, and the side rail holds community context. The illustrated map now has a dedicated place panel; mobile stacks it below the map.

Little rituals, real student knowledge.

The daily DNA capsule, optional side quests and private pocket remain. Students can now pin authenticated field notes to campus buildings. Votes, replies, saved finds and field notes use the API and survive reloads.

For the plot. And the people.

A creative common room built around useful student-to-student knowledge.

01

An unofficial campus feed

The common room replaces the oversized overview panel with readable posts, author handles, flairs, image thumbnails and a compact composer. A pinned community note and contributor karma add a student-run feel.

02

Helpful votes, replies and a private pocket

The voting rail uses existing peer-verification records. Students can mark useful posts, open their replies, save finds and copy share links. Counts and saved state come from the server.

03

Channels for every campus rabbit hole

Exam survival, After class, Future you, What's happening and Find your people organize real posts. Buzzing, Fresh and Most helpful sorts query the API, and paginated results stay bounded.

04

A field guide with breathing room

The map keeps its original building shapes and adds paper-grid illustration, numbered places, persistent selection and a usable directory. A single details panel fits beside it on desktop and below it on mobile.

05

Leave a little local lore

Students can pin field notes to a building using its existing authenticated history endpoint. Notes show the server-selected author and date and remain after reload. Wi-Fi and quiet-spot details have readable accordions.

06

Keep the tiny campus rituals

The daily DNA capsule, gold confetti, optional side quests, saved pocket, mood forecast and shuffled event feed remain. Sticky notes, doodles and gold details carry the personality through the cleaner layout.

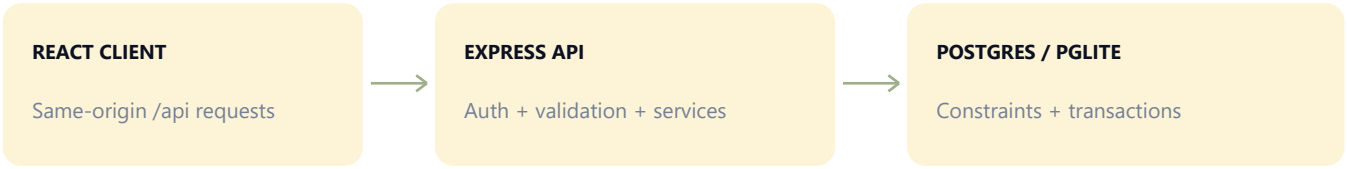
07

Keep the conversations dependable

Contribution forms, comments, RSVPs and protected removal remain connected to the backend. Search links open the selected place. Cancelled response parsing now propagates correctly, preventing a feed render failure.

A backend you can reason about.

The existing Express/PostgreSQL architecture was strengthened rather than replaced.



Authenticated identity on every write

Guest writes and fallback user IDs were removed. The server chooses the author identity. Ownership and moderator checks protect removal, and students cannot verify their own contributions.

Validated inputs and predictable queries

Zod bounds text, IDs, page sizes, categories, dates and URLs. SQL values remain parameterized; sort expressions are allowlisted and search wildcard characters are treated literally.

Atomic writes and consistent reputation

Contributions, comments, verification counts and reputation changes use transaction callbacks. Verification locks the contribution row to serialize competing toggles. Rank updates target the affected user.

A clearer security baseline

JWTs pin HS256, issuer and audience; new passwords use bcrypt cost 12. Production rejects weak/missing secrets. Helmet, an origin allowlist, a 1 MB JSON limit, API limits and separate sign-in limits are configured.

Operational visibility and safer failures

Request IDs connect client errors with server logs. Health checks return 503 when the database is unavailable. Unexpected database messages and stack traces are not returned to clients. Startup and shutdown are separate from the testable app.

A self-contained local environment

PGlite runs PostgreSQL locally with persistent files for the demo. It is explicitly selected and disabled in production. Standard PostgreSQL remains available through the existing pg adapter and environment configuration.

The right tools, for clear reasons.

Versions below reflect the installed and tested dependency lockfiles at delivery.

Layer	Technology	Why it is here
Interface	React 19.2.8 + React Router 7.18.2	Retains the existing component model and route structure. Context coordinates account, shared data, theme and contribution state.
Design	Tailwind CSS 4.3.3 + custom CSS + Lucide + React Joyride	Keeps the existing theme and guided tour. Scoped common-room styles define readable post hierarchy, responsive map columns, warm paper surfaces and playful gold details.
Frontend tooling	Vite 8.2.1 + Oxlint + Prettier	Fast local iteration, a production build, repeatable lint checks, and consistently formatted source. The proxy removes hardcoded API hosts.
API runtime	Node.js + Express 4.22.3	Preserves the established JavaScript stack and existing route modules. App/server separation makes integration tests straightforward.
Persistence	PostgreSQL via pg 8.23.0; PGlite 0.5.10 for demo	The relational schema fits accounts, contributions, comments, bookmarks and RSVPs. The demo uses real SQL without requiring a separate database service.
Validation / security	Zod 4.6.7; bcryptjs; jsonwebtoken; Helmet; cors; express-rate-limit	Bounds untrusted inputs, hashes passwords, verifies signed sessions, sets security headers, limits origins and throttles repeated requests.
Verification	Node test runner + Playwright	API tests exercise the actual SQL and HTTP behavior. Browser tests check real student workflows and responsive layouts in Microsoft Edge.
Handover	Python + ReportLab + Poppler	Creates a reproducible PDF, then renders pages for visual inspection before delivery.

The frontend keeps the existing React stack and category pages. The common room and field guide reuse current API capabilities; they require no new service or dependency. Original gold, serif, campus and midnight elements carry through the cleaner forum layout.

Tested through the whole journey.

Checks cover correctness, state persistence, authorization, and the actual browser experience.

Check	Result	Evidence / scope
Embedded SQL API suite	14 / 14 PASS	PGlite in-memory database; real HTTP requests and SQL transactions.
Native PostgreSQL API suite	14 / 14 PASS	The same suite against the configured local PostgreSQL server, in a uniquely named temporary schema.
Browser suite	14 / 14 PASS	Microsoft Edge: existing student workflows plus common-room channel filters, sorting, persistent helpful votes, readable map panels and authenticated field notes.
Responsive checks	PASS	320, 390, 768, 1024 and 1440 px widths; dashboard, discovery, map and login; no horizontal overflow.
Frontend lint + build	PASS	Oxlint with no warnings; Vite production build completed successfully.
Dependency audits	0 advisories	npm audit for both frontend and backend at delivery; compatible patches applied and unused dependency chains removed.
Visual review	PASS	Desktop and mobile feed and map, midnight theme, login, and every final PDF page rendered and inspected.

Tests exercise failure paths too.

Invalid pagination and IDs, malformed JSON, unauthorized writes, unsafe URLs, duplicate registrations, self-verification, concurrent toggles, transaction rollback, spoofed comment names, private saves and access to removed content are covered.

Test data is isolated.

Browser tests use a separate in-memory demo server. Native PostgreSQL tests create a dedicated schema, verify that it is selected, and remove that schema afterwards. Existing campus tables and data are not used for test writes.

These checks validate the local implementation. They do not constitute a load test, a penetration test, or a deployed production availability guarantee.

Ready for your next semester.

A small runbook for reviewing the project and making it your own.

01 Start the local demo

From the repository root: **npm install**, **npm run setup**, then **npm run dev**. Dependencies are already installed in this workspace. Open **http://127.0.0.1:5173**; the API runs on port 5000. Select **Try the student demo** on the sign-in page.

02 Keep your changes

Demo changes persist under **backend/.data/campus**, which is excluded from Git. Sample events receive upcoming dates when this database is first seeded. The demo endpoint and sample accounts are restricted to explicit demo mode; production rejects that mode.

03 Use standard PostgreSQL

Configure **backend/.env** using its example. Run **npm run db:init** to create the database/schema, then **npm run start:backend** and **npm run dev:frontend**. The init command does not insert sample data. Use **npm run db:seed** only when you intentionally want the sample dataset.

04 Repeat the verification

Run **npm test**, **npm run test:postgres**, **npm run test:e2e**, **npm run lint**, and **npm run build**. Browser tests use Edge on Windows; other platforms need Playwright Chromium. See the README for details and the API reference for request examples.

05 Know the boundaries

Images are linked by URL; an upload storage service and Google SSO are not configured. The campus map is schematic and the mood forecast is decorative. The daily capsule draws from up to 100 fetched contributions; common-room and discovery feeds are paginated. External images need network access, with a local campus-photo fallback.

06 Before a public deployment

Supply real campus content and infrastructure, a durable PostgreSQL service, HTTPS, a strong JWT secret and approved origins. Rate limits currently use a process-local store; use a shared store for multiple API instances. Persistent browser sessions use Web Storage and require the usual XSS controls. No external deployment was performed.

OFFICIAL IMPLEMENTATION REFERENCES

PGlite API | Vite environment configuration | Express security guidance